

DAV Recruitment Assignment

Question-1a:

Thinking Process:

I have proceeded this question in the way they have stated since I'm new to this topic. I have observed dataset clearly; I can observe there are many missing values and some outliers in study hours. Also, for exercise frequency for some of the data points, units are mentioned. I have also tried to get use of all different kinds of plots since I was unsure about which plot can useful information.

Methodology:

Firstly, I have categorized columns into numerical and categorical since, it will be easy to distinguish their behavior. For numerical columns I have filled missing values with mean of all values and similarly I have filled missing values of categorical columns with mode. For exercise frequency I have split the string and kept only numerical part to get rid of units. I have checked data types using "*df.dtypes*". I observed that all of them matched to perfection except exercise frequency, so I changed it. Pandas were used to handle data and matplotlib is used for visualization.

Outputs:

Numerical Feature	Mean	Median
Age	20.44	20.00
Study Hours Per Day	3.62	3.50
Social media	2.57	2.50
Passive Entertainment Hours	1.82	1.80
Attendance Percentage	84.14	84.40
Sleep Hours Per Night	6.45	6.50
Exercise Frequency	3.04	3.00
Mental Health Rating	5.83	6.00
Cumulative Grade	69.61	70.50
Productivity Score	4.56	4.45
Total Screen Time	4.22	4.20

Categorical Feature	Mode
Gender	Female
Position of Responsibility	No
Diet Quality	Fair
Parental Education Level	High School
Internet Quality	Good
Extracurricular Participation	No
Sleep Debt	No

Analysis:

Histogram Analysis

The histograms show that most variables follow a reasonably normal distribution. Students generally study around 3–4 hours daily, sleep about 6–7 hours, and maintain attendance between 75% and 90%. Social media usage and screen time are slightly right-skewed, indicating a few students spend excessive time on digital activities.

Boxplot Analysis

The boxplots reveal the presence of outliers in variables such as StudyHoursPerDay, SocialMediaHours, ProductivityScore, and TotalScreenTime. Most students fall within a consistent range, but a few observations show extremely high or low behavioral patterns.

Scatter Plot Analysis

Scatter plots indicate that:

- StudyHoursPerDay, ProductivityScore, Exercise_frequency, and mental_health_rating have a positive relationship with Cumulative_Grade.
- SocialMediaHours, PassiveEntertainmentHrs, and TotalScreenTime show a negative relationship with academic performance.
- Age has little to no effect on grades.
- SocialMediaHours vs Cumulative_Grade shows a weak negative relationship, implying that excessive social media usage may reduce academic performance.
- SleepHoursPerNight vs Cumulative_Grade indicates a weak but balanced relationship, suggesting that moderate sleep contributes to stable performance.

Correlation Heatmap Analysis

The heatmap confirms that:

- *ProductivityScore* and *StudyHoursPerDay* are strongly positively correlated with *Cumulative_Grade*.
- *SocialMediaHours* and *TotalScreenTime* negatively impact grades and productivity.
- Mental health, exercise, and sleep are positively related to academic performance and productivity.

Reasonable Queries:

Does studying more improve academic performance?

Yes. The scatter plot and correlation heatmap show a strong positive relationship between *StudyHoursPerDay* and *Cumulative_Grade*. Students who study more generally achieve higher grades.

Does social media usage affect grades?

Yes. *SocialMediaHours* shows a negative correlation with *Cumulative_Grade*, indicating that excessive social media usage may reduce academic performance.

Is productivity linked to better grades?

Yes. *ProductivityScore* has one of the strongest positive correlations with *Cumulative_Grade*, suggesting highly productive students perform better academically.

Do healthy lifestyle factors influence performance?

Yes. Better *SleepHoursPerNight*, *Exercise_frequency*, and *mental_health_rating* are positively associated with higher grades and productivity.

Does total screen time impact academic success?

Yes. The plots show that higher *TotalScreenTime* is negatively correlated with both *ProductivityScore* and *Cumulative_Grade*.

Does age significantly affect student grades?

No. The scatter plot between *Age* and *Cumulative_Grade* shows no strong relationship, indicating age has minimal impact on academic performance within this dataset.

Anomalies Found:

- Most variables show near-normal distribution.
- Higher study hours improve academic performance.
- *ProductivityScore* has a strong positive correlation with grades.
- Social media usage and screen time negatively affect grades.
- Better sleep, exercise, and mental health improve productivity and performance.
- Attendance is generally high among students.

Question-1b:

Outputs:

Linear Regression:

$R^2 = 0.6972952485412818$

RMSE = 9.577243425926795

MAE = 6.995378877012439

Random Forest Regressor:

$R^2 = 0.7466775298766699$

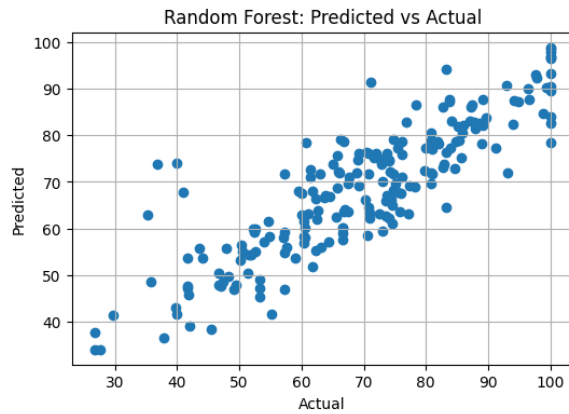
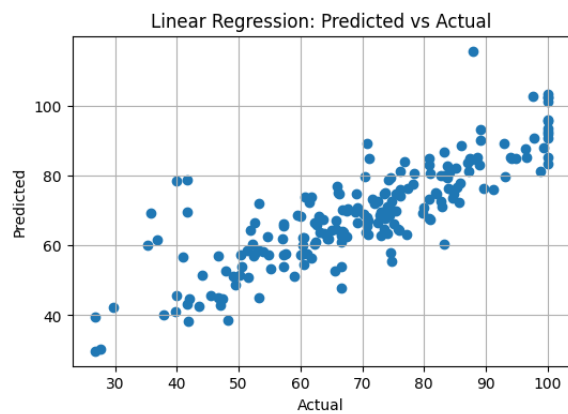
RMSE = 8.761284116676864

MAE = 6.750597014925371

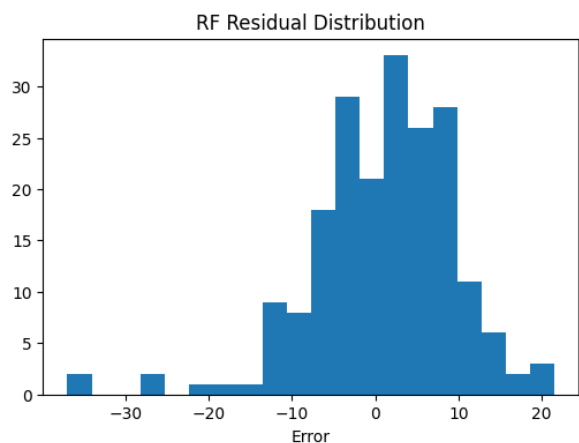
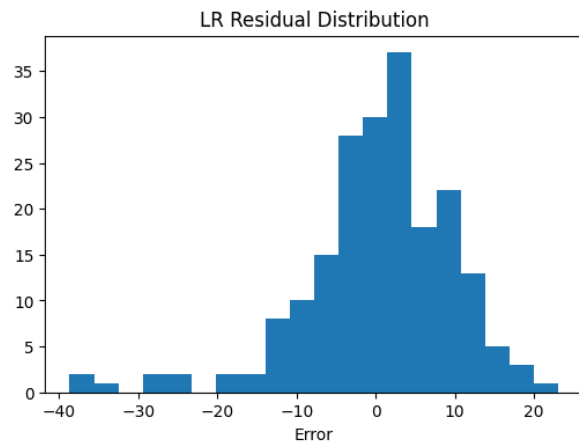
Feature	Linear Regression Coefficient	Random Forest Importance
mental_health_rating	11.462263	0.112943
Exercise_frequency	-5.096545	0.009609
StudyHoursPerDay	4.698602	0.524427
SleepHoursPerNight	-4.209661	0.031487
Gender_Other	3.155904	0.001190
internet_quality_Good	-2.337555	0.003676
internet_quality_Poor	-2.005481	0.002357
PassiveEntertainmentHrs	-1.831468	0.024180
SocialMediaHours	-1.797242	0.027731
parental_education_level_Master	-1.361710	0.003148
PoR_Yes	-0.950765	0.003064
parental_education_level_High School	-0.647593	0.004180
SleepDebt_Yes	0.487788	0.001386
ProductivityScore	-0.367721	0.156620

Feature	Linear Regression Coefficient	Random Forest Importance
Age	-0.279180	0.013313
TotalScreenTime	0.260934	0.038693
Gender_Male	0.148265	0.003668
AttendancePercentage	0.145846	0.027736
Diet_Quality_Poor	0.143018	0.003300
Diet_Quality_Good	0.133795	0.004058
extracurricular_participation_Yes	-0.019059	0.003235

Prediction vs True Labels:



Error Plots:



Comment on model performance:

- The Random Forest Regressor performs better than the Linear Regression model.
- It achieves a higher R^2 score (0.7467), meaning it explains more variance in student grades.
- It also has lower RMSE and MAE values, indicating smaller prediction errors.
- Linear Regression captures only linear relationships, whereas Random Forest can model complex and non-linear interactions between variables.
- Therefore, Random Forest provides more accurate and reliable predictions for this dataset.
- Both models capture the overall trend well.
- Random Forest predictions are closer to actual values than Linear Regression.
- Linear Regression shows higher spread and larger prediction errors.
- Random Forest handles complex patterns more effectively.
- Residuals of both models are centered near zero.
- Random Forest has smaller and more concentrated residuals, indicating better accuracy.
- A few outliers are present in both models.

Analysis on top features:

Top features according to random forest regressor are:

1. StudyHoursPerDay
2. ProductivityScore
3. mental_health_rating
4. TotalScreenTime
5. SleepHoursPerNight

StudyHoursPerDay

StudyHoursPerDay is the most important predictor because academic performance strongly depends on consistent learning and practice. Students who spend more time studying are likely to revise concepts more effectively, complete assignments on time, and perform better in exams. The high importance value (0.524) is intuitive because study time directly contributes to knowledge acquisition and exam preparation.

ProductivityScore

ProductivityScore is another major predictor since it reflects how efficiently students manage their time and academic tasks. A productive student may achieve better outcomes even without extremely long study hours because of better concentration, planning, and learning strategies. Its relatively high importance makes sense because academic success depends not only on effort but also on the effectiveness of that effort.

Question-2:

Ideation:

Basically, I don't have previous knowledge about different models, so I decided to experiment on some of the famous machine learning models. I have picked linear regression as a base model and logistical regression, random forest regressor, decision tree and SVM as my primary models. I have built basic understanding on these models and used them. Rather than depending on accuracy metric, I have decided to use Confusion Matrices, Macro/Micro F1-Scores, Precision and Recall as secondary evaluation metrics

Methodology:

Since I have taken linear regression as my ground, I have firstly observed its predicted vs actual plot and residual distribution plot. For that what I have observed is The Linear Regression model achieved a low R^2 score (~ 0.18), indicating weak predictive performance. The RMSE (~ 1.03) and MAE (~ 0.88) show moderate prediction error.

The Predicted vs Actual plot shows that the model struggles to clearly separate the target classes because Linear Regression is mainly suited for continuous data rather than classification tasks.

The residuals are roughly centered around zero, but the spread indicates noticeable prediction errors.

Coefficient analysis shows that features such as Profession_Healthcare positively influence predictions, while features like Profession_Entertainment negatively affect them. Variables such as Age and Work_Experience have very little impact.

Overall, Linear Regression is not highly effective for this categorical prediction problem. So, after looking at these issues decided to use a model which overcomes the drawbacks faced by linear regression model.

Even though scaling is not necessary for linear regression and random forest, since it is necessary for logistical for other models I have implemented scaling such that mean of numerical columns is 0 and standard deviation is 1.

Categorical variables were converted using one-hot encoding, and missing values were handled using median and mode imputation. Multiple models including Logistic Regression, Decision Tree, Random Forest, and SVM were trained and compared using accuracy and F1-score.

Logistic Regression and Random Forest showed the best performance. Feature importance analysis indicated that Age, Work_Experience, and Family_Size were the most influential features. Overall, preprocessing, model evaluation, and feature analysis were performed to build and compare classification models.

Comparison between different models:

- **Logistic Regression** achieved the highest accuracy (~51%) and showed the most balanced classification performance overall.
- **Random Forest** performed similarly to Logistic Regression and produced stable predictions with strong handling of important features.
- **Decision Tree** showed lower generalization ability and higher misclassification rates.
- **SVM** performed the worst, particularly struggling to correctly classify Class 1 samples.

<i>Model</i>	<i>Accuracy</i>	<i>Macro F1 Score</i>	<i>Weighted F1 Score</i>	<i>Performance Summary</i>
Logistic Regression	0.5099	0.4847	0.4983	Best overall accuracy with balanced performance across most classes.
Random Forest	0.4981	0.4848	0.4962	Performance very close to Logistic Regression with stable class predictions.
Decision Tree	0.4535	0.4457	0.4558	Lower accuracy and less stable predictions compared to ensemble methods.
SVM	0.4455	0.3839	0.4025	Weakest overall performance, especially poor recall for some classes.

Since Logistical regression has slight edge over remaining models, I have decided to use it for labelling in 2nd dataset

How Logistical Regression outperformed others:

Among all the classification models tested, Logistic Regression achieved the best overall performance with the highest accuracy and competitive F1-score values. It provided more balanced predictions across different classes compared to Decision Tree, Random Forest, and SVM. While Random Forest produced similar results, Logistic Regression showed slightly better generalization and stability on the test dataset. SVM and Decision Tree struggled with class separation and produced lower overall metrics.

Question-4:

Firstly, I have decided to learn basics of selenium library and I have also done that.

First problem I have faced is login issue. Means how can I automate login. So, I have made program to wait until I login. I must press enter in python terminal to proceed further. So, login issue solved.

Next, some of the viewable items are unable to be clicked, so I implemented force click function which passes through all interaction layers and dropdowns and clicks wanted button.

ASC portal is divided into multiple frames. Switching between frames fluently has become an issue. So, when I wanted to switch frame, I have restarted by going to default view and changing the frame.

While selecting departments, they are encoded in zebra striped boxes. So, when I am using XPATH including bgcolor, only alternative departments are being selected but once I omit bgcolor every cell is being selected including headers and footers. So, I have decided to start iteration with 4th on so that headings, headers will be skipped.

At last, when I am trying to encode data in excel, I have found that there are some illegal symbols in the data. So, I cleansed data with the code given by ChatGPT.

```
import re

def clean_excel_text(value):

    if isinstance(value, str):
        return re.sub(r'[\x00-\x1F\x7F]', '', value)

    return value

sheet_1 = sheet_1.map(clean_excel_text)
sheet_2 = sheet_2.map(clean_excel_text)
```

Also, I have joined courses I first 5 branches for simplicity and to keep them in single sheet.

I have used Edge for opening the website. And extracting completing data is taking around 15 minutes in my laptop.

Most of the times I have used relative XPATHS for the program to work even the website layout changes.

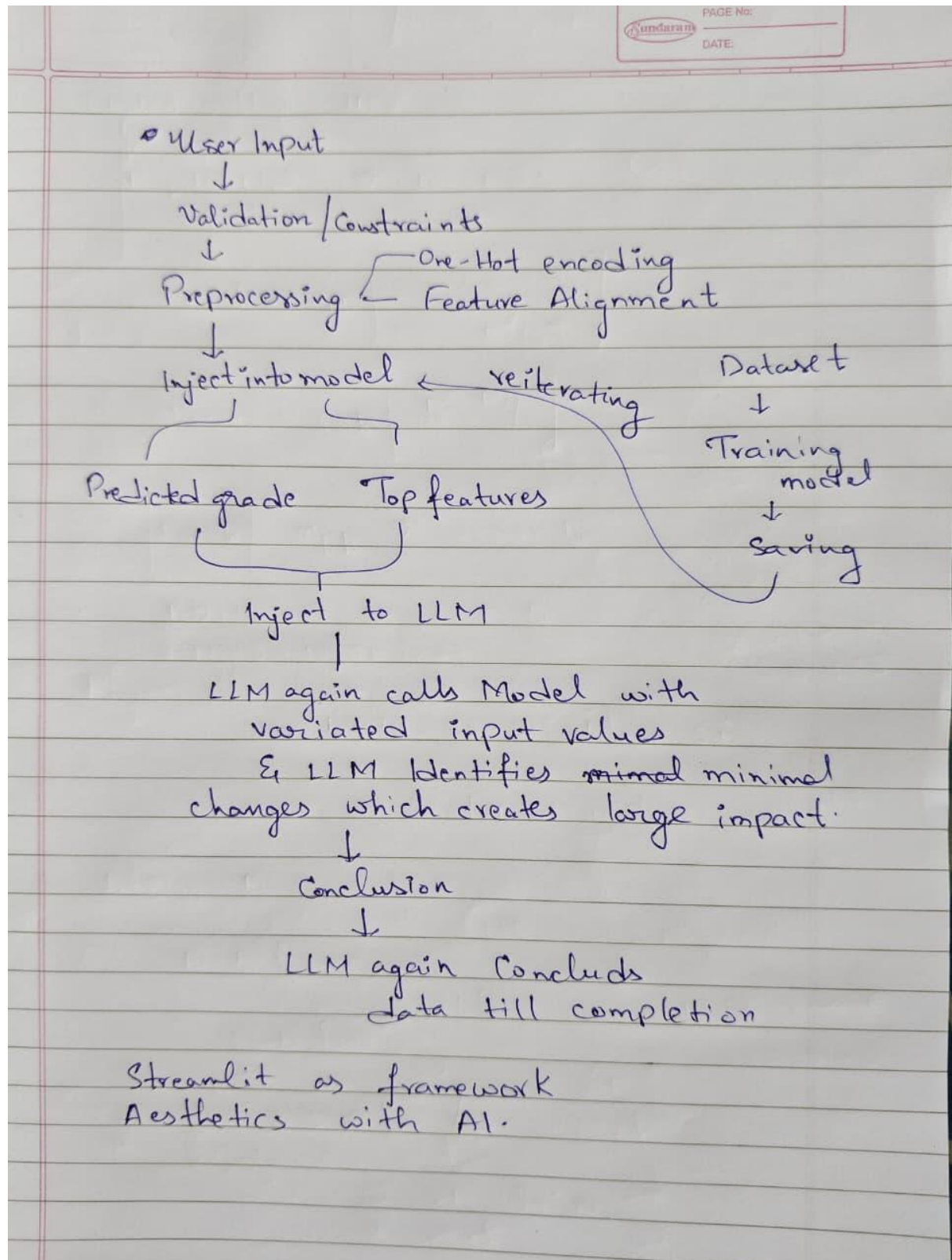
The ASC portal used nested HTML frames, requiring repeated frame switching after navigation events.

Certain menu elements were dynamically rendered and not directly interactable using standard Selenium click operations.

Finally, I have used pandas to handle the data.

Bonus Question:

Concept Sketch:



System Overview:

My website is designed to predict a student's cumulative academic grade based on the features that have been given in the sample dataset. After predicting the performance score, the system generates a personalised recommendation using a Large Language Model (LLM).

The system works in three major stages:

1. Input collection from the student
2. Machine learning prediction
3. AI-generated recommendation

Input to the system:

The system accepts multiple student-related features from the user interface. And converts these inputs into a json to feed into working program. Input columns given to json are:

- Age
- Gender
- StudyHoursPerDay
- SocialMediaHours
- PassiveEntertainmentHrs
- AttendancePercentage
- SleepHoursPerNight
- Diet_Quality
- Exercise_frequency
- parental_education_level
- internet_quality
- mental_health_rating
- extracurricular_participation
- PoR

I thought to make UI interactive by making numerical columns working with slider and categorial columns to be worked with dropdown list.

Transformation Pipeline

The following preprocessing operations are performed before prediction:

1. Missing value handling
 - Numerical columns are filled using mean values.
 - Categorical columns are filled using mode values.
2. Data type cleaning
 - Exercise frequency values are converted into numeric form.
3. One-hot encoding
 - Categorical features are converted into machine-readable binary columns using `pd.get_dummies()`.
4. Feature alignment
 - Incoming user data is reindexed to match the exact feature columns used during training.
5. Prediction
 - The processed data is passed into a trained Random Forest Regressor model.
6. Recommendation generation
 - The predicted cumulative grade and important student features are sent to an LLM through the Groq API. **Output of the System**

The system returns:

- Predicted cumulative grade (out of 100)
- Performance status indicator
- Top influential features
- AI-generated personalised recommendation

Why choose that model?

Model chosen is **Random Forest Regressor**

Random Forest Regressor was selected because: It can capture nonlinear relationships between student habits and grades.

- It is resistant to overfitting compared to a single decision tree.
- It automatically estimates feature importance.
- It handles complex behavioural interactions effectively.

Since student performance depends on many interacting factors such as study hours, sleep, attendance, and screen time, Random Forest provides better flexibility than Linear Regression.

Prompt Engineering:

Exact Prompt Used

You are an academic advisor. Address the student directly as "you".

Predicted cumulative grade: {cgpa:.2f}/100

Student profile (current values):

{feature_text}

Task:

- 1) Perform a brief sensitivity check: slightly vary each listed feature (increase/decrease within realistic bounds) and determine which single change most improves cumulative grade.
- 2) Identify the MOST impactful feature for improvement.
- 3) Also identify TWO other helpful features.

Output rules:

- Write a short description (3–4 sentences total).
- First sentence should clearly state which improvement most increases performance.
- Next sentences should give actionable suggestions for two additional features.
- Use direct and practical language.
- Do not mention model names, feature importance, or sensitivity analysis.
- Do not use bullet points or JSON formatting.

Style:

- Concrete and actionable
- Concise and student-friendly

Justification for Prompt Design:

1. Direct Student Interaction

The prompt instructs the model to address the student directly using “you” to make recommendations more personal and engaging.

2. Controlled Output Length

The response is restricted to 3–4 sentences to:

- reduce verbosity
- improve readability
- maintain UI cleanliness

3. Actionable Recommendations

The prompt specifically requests practical suggestions such as:

- increasing study hours
- improving sleep schedule
- reducing social media usage

This avoids vague motivational advice.

4. Restricting Technical Terms

The LLM is instructed not to mention:

- machine learning
- feature importance
- sensitivity analysis

This keeps the output understandable for normal users.

5. Dynamic Feature Injection

The prompt dynamically injects:

- predicted grade
- top influential features
- actual feature values

This makes every recommendation personalised. And above prompt is adjusted many time by different set of trails.

Failure Modes:

Failure Mode 1: Misleading Recommendations

Problem

The LLM may generate unrealistic advice such as:

- Increase your age
- recommendations based on non-actionable features

Mitigation

Non-actionable features such as:

- Age
- Student_ID

were filtered from recommendation generation.

Failure Mode 2: Invalid User Inputs

Problem

Users may enter unrealistic values:

- negative study hours
- attendance above 100

Mitigation

Client-side validation was implemented using:

- sliders
- bounded numerical ranges
- dropdown menus

Failure Mode 3: LLM API Failure or Quota Exhaustion

Problem

The external API may fail because of:

- quota exhaustion
- internet issues
- API downtime

Mitigation

The system includes exception handling and fallback recommendations to ensure the application still functions even if the LLM service becomes unavailable.

Changes implemented after initial trails:

Initially, the LLM recommendations were too generic and occasionally suggested unrealistic improvements such as increasing age. After observing these outputs, the prompt and preprocessing pipeline were modified to:

- remove non-actionable features
- improve feature formatting
- enforce concise actionable recommendations

This significantly improved the relevance and realism of the generated advice.

Other Information:

Basic UI elements in the website are kept by experimenting in Figma and asking AI to give CSS code to implement those features.

Interface.py is the file need to be run in Github repo for the app to run.

From the trained model data is exported and imported into advisor.py. Model training can be seen in Jupyter notebook.

Link for testing the app:

<https://ai-advisor-bkzrksawueaxstegi7v656.streamlit.app/>